
RAJSHAHI UNIVERSITY OF ENGINEERING &
TECHNOLOGY

Department of Computer Science & Engineering

Numerical Methods Laboratory Manual

Modeling, Computers & Error Analysis

Chapter 1 · Foundations of Numerical Methods

Prepared by: Md. Azmain Yakin Srizon
Assistant Professor
Dept. of Computer Science & Engineering, RUET

Topics: Mathematical Modeling & Euler's Method,
Approximations and Errors, Taylor & Maclaurin Series,
Finite-Difference Derivatives, and
Round-off Errors & Machine Epsilon

Reference: S.C. Chapra & R.P. Canale, *Numerical Methods
for Engineers*, McGraw-Hill — Part One (Chs. 1–4)

Contents

1	Introduction and How to Use This Manual	2
2	Lab Report: Format and Submission Guidelines	2
2.1	Report structure	2
2.2	Formatting rules	3
2.3	Submission rules	3
3	Experiment 1: Mathematical Modeling & Euler's Method	4
3.1	Theory	4
3.2	Worked Example (full input and output)	4
4	Experiment 2: Approximations, Errors & the Stopping Criterion	6
4.1	Theory	6
4.2	Worked Example (full input and output)	6
5	Experiment 3: Taylor & Maclaurin Series Approximation	8
5.1	Theory	8
5.2	Worked Example (full input and output)	8
6	Experiment 4: Finite-Difference Approximation of Derivatives	10
6.1	Theory	10
6.2	Worked Example (full input and output)	10
7	Experiment 5: Round-off Errors & Machine Epsilon	12
7.1	Theory	12
7.2	Worked Example (full input and output)	12
	Appendix: Quick Reference of the Foundations	14

1 Introduction and How to Use This Manual

This manual accompanies the lecture material on **Chapter 1: Modeling, Computers, and Error Analysis** — the foundations on which every later method rests. Each experiment treats one *computational technique* and is organised uniformly:

- **Objective** — what the experiment teaches.
- **Theory** — a short statement of the method and its error behaviour.
- **Algorithm** — complete *pseudocode* you must implement.
- **Worked Example** — a fully solved example showing the *exact input* and the *complete output* (results table and final value).
- **Exercise** — at least ten problems to be solved and reported.

Convention. The true (exact) value is x_{true} and an approximation $\tilde{x}$. Then the **true error** $E_t = x_{\text{true}} - \tilde{x}$, the **true relative error** $\varepsilon_t = |E_t/x_{\text{true}}| \times 100\%$, and the **approximate relative error** $\varepsilon_a = |(\tilde{x}_{\text{new}} - \tilde{x}_{\text{old}})/\tilde{x}_{\text{new}}| \times 100\%$. A result is accepted to n significant figures once $|\varepsilon_a| < \varepsilon_s = (0.5 \times 10^{2-n})\%$. Tabulate *every* iteration/step and report values to the precision requested.

A general-purpose driver (in C/C++/Python/MATLAB) is recommended: read the function (and its derivative or slope where required), the starting data and the step/tolerance, then loop until the stopping test is met, printing one row of the results table each pass.

2 Lab Report: Format and Submission Guidelines

Every submitted report **must** follow the structure and rules below.

2.1 Report structure

1. **Cover page** — Course title and code, Experiment number and name, your full name, roll/registration number, section, group, date of performance, date of submission.
2. **Objective** — one or two lines.
3. **Theory** — the method, its formula and error behaviour (in your own words; 4–6 lines).
4. **Algorithm / Flowchart** — the pseudocode (or a flowchart) you implemented.
5. **Source code** — complete, commented program listing.
6. **Sample input** — the function/data, starting values and step/tolerance you used.
7. **Output / Results** — the *full results table* (steps, estimates and errors) and the final value. Paste the program output *and* present a neat hand-formatted table.
8. **Discussion** — number of steps/terms, observed order of the error, effect of step size h (and round-off at very small h), and any difficulty faced.
9. **Conclusion** — 2–3 lines.

2.2 Formatting rules

- Paper size **A4**; margins **2.5 cm** on all sides.
- Body font **12 pt** serif (Times/Latin Modern); line spacing **1.5**; justified text.
- Headings bold; code in a **monospaced** font (e.g. Consolas/Courier), 10–11 pt, with line numbers.
- Every table and figure must be **numbered and captioned**. Tabulate results with aligned decimal points.
- Number all equations you reference.

2.3 Submission rules

- Submit a single **PDF** named `Roll_ExpNo_MethodName.pdf` — for example, `2303001_Exp01_Euler-Method.pdf`.
- Submit the **source files** separately in a folder/zip with the same base name.
- Deadline: **within two weeks** of performing the experiment unless announced otherwise. Late reports lose marks as per course policy.
- Work must be **your own**. Copied reports/code receive zero; both provider and copier are penalised. You may discuss ideas but must write and code independently.

Marking (typical). Algorithm & code correctness 40%, results & error tables 25%, discussion 20%, formatting & neatness 10%, viva/demo 5%.

3 Experiment 1: Mathematical Modeling & Euler's Method

Objective

To turn a physical law into a differential-equation model and solve it *numerically* by Euler's method, comparing with the analytical solution and studying how the error depends on the step size h .

3.1 Theory

Applying Newton's second law to a falling body with linear air drag gives the model

$$\frac{dv}{dt} = g - \frac{c}{m} v, \quad v(0) = 0.$$

Euler's method replaces the derivative by a slope over a finite step and marches forward:

$$v_{i+1} = v_i + \phi(t_i, v_i) h, \quad \phi = \left. \frac{dv}{dt} \right|_i = g - \frac{c}{m} v_i.$$

It is first order: the global error at a fixed time is $O(h)$, so *halving* h roughly halves the error.

Algorithm 1 Euler's Method for a First-Order ODE

Input: slope function $\phi(t, v)$, initial t_0, v_0 , step h , end time t_{end}

Output: table of (t, v)

1: $t \leftarrow t_0$; $v \leftarrow v_0$; **print** t, v

2: **while** $t < t_{\text{end}}$ **do**

3: $v \leftarrow v + \phi(t, v) \cdot h$

▷ Euler step

4: $t \leftarrow t + h$

5: **print** t, v

6: **end while**

7: **return** v

3.2 Worked Example (full input and output)

Example 1.1 — the falling parachutist

Input. $\phi(t, v) = g - \frac{c}{m}v$ with $g = 9.8$, $c = 12.5$ kg/s, $m = 68.1$ kg; $v(0) = 0$, $h = 2$ s.
Analytical solution $v(t) = \frac{gm}{c}(1 - e^{-(c/m)t})$, terminal velocity $\frac{gm}{c} = 53.39$ m/s.

Output.

t (s)	Euler v	Analytical v	$ \varepsilon_t $
0	0.00	0.00	—
2	19.60	16.40	19.5%
4	32.00	27.77	15.2%
6	39.86	35.64	11.8%
8	44.82	41.10	9.1%
10	47.97	44.87	6.9%
∞	—	53.39	—

Sample step: $v(2) = 0 + [9.8 - 0] 2 = 19.60$; $v(4) = 19.60 + [9.8 - \frac{12.5}{68.1}(19.60)] 2 = 32.00$.

Euler *over-shoots*; with $h = 1$ the errors roughly halve. $v \rightarrow 53.39$ m/s (terminal).

Exercise 1 (Euler's Method)

Solve each IVP by Euler's method, tabulate every step, and (where an exact solution is given) report ε_t .

1. Parachutist model above with $h = 1, 0.5, 0.25$; show the error at $t = 10$ roughly halves as h halves.
2. $\frac{dy}{dx} = -y$, $y(0) = 1$ on $[0, 1]$, $h = 0.1$; compare with e^{-x} .
3. $\frac{dy}{dx} = x + y$, $y(0) = 1$, $h = 0.1$; exact $y = 2e^x - x - 1$.
4. $\frac{dy}{dx} = y - x^2 + 1$, $y(0) = 0.5$, $h = 0.2$; exact $y = (x + 1)^2 - 0.5e^x$.
5. $\frac{dy}{dx} = -2xy$, $y(0) = 1$, $h = 0.1$; exact e^{-x^2} .
6. Nonlinear drag $\frac{dv}{dt} = 9.8 - 0.25v^2$, $v(0) = 0$, $h = 0.5$; estimate the terminal velocity.
7. Newton's cooling $\frac{dT}{dt} = -0.07(T - 20)$, $T(0) = 90$, $h = 2$; compare with the exact decay.
8. RC circuit $\frac{dq}{dt} = \frac{1}{R}(V - \frac{q}{C})$, $R = 1$, $C = 1$, $V = 5$, $q(0) = 0$, $h = 0.1$.
9. Logistic growth $\frac{dp}{dt} = 0.8p(1 - \frac{p}{10})$, $p(0) = 1$, $h = 0.25$.
10. Free fall with no drag $\frac{dv}{dt} = 9.8$, $v(0) = 0$; verify Euler is *exact* here (why?).
11. Tabulate the global error of Exercise 2 at $x = 1$ for $h = 0.2, 0.1, 0.05, 0.025$ and confirm it is $O(h)$.
12. Implement the slope as a separate *function* so the same driver solves Exercises 2–9 unchanged.

4 Experiment 2: Approximations, Errors & the Stopping Criterion

Objective

To quantify accuracy with the true error, the true and approximate relative errors and significant figures, and to implement the iterative stopping rule on a convergent series.

4.1 Theory

$E_t = x_{\text{true}} - \tilde{x}$, $\varepsilon_t = |E_t/x_{\text{true}}| \times 100\%$, $\varepsilon_a = |(\tilde{x}_{\text{new}} - \tilde{x}_{\text{old}})/\tilde{x}_{\text{new}}| \times 100\%$. When the true value is unknown, iterate until $|\varepsilon_a| < \varepsilon_s$, where

$$\varepsilon_s = (0.5 \times 10^{2-n})\%$$

guarantees the result to n significant figures.

Algorithm 2 Series Summation with a Stopping Criterion (e.g. e^x)

Input: x , number of significant figures n

Output: $sum \approx e^x$ correct to n significant figures

1: $\varepsilon_s \leftarrow 0.5 \times 10^{2-n}$

2: $sum \leftarrow 0$; $term \leftarrow 1$; $k \leftarrow 0$

3: **repeat**

4: $sum \leftarrow sum + term$

5: $\varepsilon_a \leftarrow |term/sum| \times 100$

$\triangleright |\text{new} - \text{old}| = |\text{term}|$

6: $k \leftarrow k + 1$; $term \leftarrow term \cdot x/k$

7: **until** $\varepsilon_a < \varepsilon_s$

8: **return** sum

4.2 Worked Example (full input and output)

Example 2.1 — absolute vs. relative error (bridge & rivet)

Input. A bridge (true 10000 cm) is measured as 9999 cm; a rivet (true 10 cm) as 9 cm.

Output. Both have $E_t = 1$ cm, but $\varepsilon_t^{\text{bridge}} = \frac{1}{10000} \times 100\% = \boxed{0.01\%}$ while $\varepsilon_t^{\text{rivet}} = \frac{1}{10} \times 100\% = \boxed{10\%}$ — the relative error, not the absolute, measures quality.

Example 2.2 — $e^{0.5}$ to three significant figures

Input. $e^x = 1 + x + \frac{x^2}{2!} + \dots$ at $x = 0.5$ (true 1.648721); stop at $\varepsilon_s = 0.05\%$ ($n = 3$).

Output.

terms	estimate	ε_t	ε_a
1	1.000000	39.3%	—
2	1.500000	9.02%	33.3%
3	1.625000	1.44%	7.69%
4	1.645833	0.175%	1.27%
5	1.648438	0.0172%	0.158%
6	1.648698	0.0014%	0.0158%

At **6 terms** $\varepsilon_a = 0.0158\% < \varepsilon_s = 0.05\%$, so we stop: $e^{0.5} \approx \boxed{1.6487}$ (good to better than 3 sig. figs). Note ε_a stays *above* ε_t — a safe bound.

Exercise 2 (Errors & Stopping)

1. Evaluate $e^{0.5}$ to 3 significant figures; reproduce the table above.
2. Evaluate e^1 to 4 significant figures; how many terms are needed?
3. Evaluate $\sin(\pi/6)$ from $\sin x = x - \frac{x^3}{3!} + \dots$ to 3 sig. figs.
4. Evaluate $\cos(0.3)$ from $\cos x = 1 - \frac{x^2}{2!} + \dots$ to 4 sig. figs.
5. Evaluate $\ln(1.2)$ from $\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \dots$ to 3 sig. figs.
6. Given $x_{\text{true}} = 2.718282$ and $\tilde{x} = 2.716667$, compute E_t and ε_t .
7. State ε_s for $n = 1, 2, 3, 4, 5$ significant figures.
8. Estimate $\sqrt{2}$ by $x_{k+1} = \frac{1}{2}(x_k + 2/x_k)$, $x_0 = 1$; tabulate ε_a until 4 sig. figs.
9. For the $e^{0.5}$ series, tabulate ε_a and ε_t together and confirm $\varepsilon_a > \varepsilon_t$ at every step.
10. Approximate π by Leibniz' series $4(1 - \frac{1}{3} + \frac{1}{5} - \dots)$; how many terms for 2 sig. figs? Comment on the slow convergence.
11. A measured length $\tilde{x} = 24.5$ cm has ± 0.05 cm tolerance; give its relative error and its number of significant figures.
12. Modify the stopping algorithm to also *cap* the number of terms at N_{max} and report a warning if convergence is not reached.

5 Experiment 3: Taylor & Maclaurin Series Approximation

Objective

To approximate a function near a base point by its Taylor series order by order, and to estimate the truncation error from the remainder term.

5.1 Theory

$$f(x_{i+1}) = \sum_{k=0}^n \frac{f^{(k)}(x_i)}{k!} h^k + R_n, \quad R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!} h^{n+1} = O(h^{n+1}),$$

with $h = x_{i+1} - x_i$. The *zero*-order term is a constant, the *first* adds the slope, and so on; expanded about $x_i = 0$ it is the **Maclaurin** series. The truncation error falls by 2^{n+1} when h is halved.

Algorithm 3 Taylor-Series Approximation to Order n

Input: derivatives $f^{(k)}(x_i)$ for $k = 0, \dots, n$, base x_i , step h

Output: $approx \approx f(x_i + h)$

1: $approx \leftarrow 0$; $hp \leftarrow 1$

2: **for** $k = 0$ **to** n **do**

3: $approx \leftarrow approx + \frac{f^{(k)}(x_i)}{k!} hp$

4: $hp \leftarrow hp \cdot h$

5: **end for**

6: **return** $approx$

5.2 Worked Example (full input and output)

Example 3.1 — a polynomial, order by order

Input. $f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$; predict $f(1)$ from $x_i = 0$, $h = 1$ (true $f(1) = 0.2$).

Output.

order	term added	$f(1) \approx$	E_t
0	$f(0) = 1.2$	1.20	-1.00
1	$f'(0)h = -0.25$	0.95	-0.75
2	$\frac{f''(0)}{2!}h^2 = -0.50$	0.45	-0.25
3	$\frac{f'''(0)}{3!}h^3 = -0.15$	0.30	-0.10
4	$\frac{f^{(4)}(0)}{4!}h^4 = -0.10$	0.20	0.00

The **fourth**-order Taylor series is exact for a quartic: $R_4 = 0$ because all higher derivatives vanish.

Example 3.2 — $\cos(\pi/3)$ from $\cos(\pi/4)$

Input. Base $x_i = \pi/4$, $h = \pi/12 \approx 0.2618$, true $\cos(\pi/3) = 0.5$; $f' = -\sin$, $f'' = -\cos, \dots$

Output.

order	estimate	ε_t
0	0.707107	-41.4%
1	0.521987	-4.40%
2	0.497754	0.449%
3	0.499869	0.0262%
4	0.500008	-0.0015%

A transcendental function needs *infinitely* many terms to be exact, but the error falls by about an order of magnitude per term ($h < 1$).

Exercise 3 (Taylor & Maclaurin Series)

1. Expand e^x about $x = 0$ to four terms; estimate $e^{0.1}$ and bound R_3 .
2. Reproduce Example 3.2 ($\cos(\pi/3)$ from $\pi/4$); then redo it from $x_i = \pi/6$ and compare the errors.
3. Expand $\ln x$ about $x = 1$; estimate $\ln(1.1)$ to four orders.
4. Expand $\sqrt{x}$ about $x = 4$; estimate $\sqrt{3.8}$ and $\sqrt{4.2}$.
5. How many Maclaurin terms give $\sin(0.5)$ correct to 4 decimal places?
6. For $f(x) = x^3 - 2x + 1$, show the third-order Taylor series about any point is exact.
7. Estimate the truncation error of a *second*-order Taylor series for $e^{0.2}$ and compare with the actual error.
8. Expand $\tan x$ about $x = 0$ to three terms; estimate $\tan(0.2)$.
9. Use a first-order Taylor expansion to derive the forward-difference derivative formula and state its error order.
10. Expand $(1 + x)^{1/2}$ (binomial/Taylor); estimate $\sqrt{1.05}$.
11. Compare the truncation error predicted by R_n with the measured error for the order-2 estimate of $\cos(\pi/3)$.
12. Plot (or tabulate) the order-1, 2, 3 Taylor approximations of e^x near $x = 0$ and observe convergence.

6 Experiment 4: Finite-Difference Approximation of Derivatives

Objective

To approximate the first derivative by forward, backward and centered finite differences obtained from the Taylor series, and to verify their error orders $O(h)$ and $O(h^2)$.

6.1 Theory

$$\underbrace{f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{h}}_{\text{forward, } O(h)}, \quad \underbrace{f'(x_i) \approx \frac{f(x_i) - f(x_{i-1}))}{h}}_{\text{backward, } O(h)}, \quad \underbrace{f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}}_{\text{centered, } O(h^2)}.$$

The centered formula is one order more accurate (its leading error terms cancel by symmetry), so halving h cuts its error *four*-fold.

Algorithm 4 Finite-Difference First Derivative

Input: function f , point x , step h

Output: forward, backward and centered estimates of $f'(x)$

- 1: $fwd \leftarrow (f(x+h) - f(x))/h$
- 2: $bwd \leftarrow (f(x) - f(x-h))/h$
- 3: $cen \leftarrow (f(x+h) - f(x-h))/(2h)$
- 4: **return** fwd, bwd, cen

6.2 Worked Example (full input and output)

Example 4.1 — $f'(0.5)$ and the effect of h

Input. $f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$ at $x = 0.5$; true $f'(0.5) = -0.9125$. Try $h = 0.5$ and $h = 0.25$.

Output.

h	forward	ε_t	backward	ε_t	centered	ε_t
0.50	-1.45000	58.9%	-0.55000	39.7%	-1.00000	9.6%
0.25	-1.15469	26.5%	-0.71406	21.7%	-0.93438	2.4%

Halving h roughly *halves* the forward/backward error ($O(h)$) but *quarters* the centered error ($O(h^2)$): the centered estimate -0.934 is by far the best.

Exercise 4 (Finite Differences)

1. Re-do Example 4.1 with $h = 0.1$; compare all three formulas to the true -0.9125 .
2. $f(x) = e^x$: estimate $f'(1)$ with $h = 0.1$ and 0.05 ; verify the centered error quarters.
3. $f(x) = \sin x$: estimate $f'(\pi/4)$ with $h = 0.1$; true value $\cos(\pi/4)$.
4. $f(x) = \ln x$: estimate $f'(2)$ with $h = 0.1$; true value 0.5 .
5. Second derivative $f''(x_i) \approx \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2}$: apply to $f = x^3$ at $x = 2$, $h = 0.1$.
6. From the position data $s(t)$: $(0, 0), (1, 0.7), (2, 1.8), (3, 3.4), (4, 5.5)$, estimate the velocity at $t = 2$ by a centered difference.
7. Tabulate the forward-difference error of $f(x) = e^x$ at $x = 0$ for $h = 0.2, 0.1, 0.05, 0.025$ and confirm it is $O(h)$.
8. Compare forward and centered differences for $f(x) = x^3$ at $x = 2$ with $h = 0.1$.
9. Apply *Richardson extrapolation* to two centered estimates (h and $h/2$) of $f'(1)$ for e^x to obtain an $O(h^4)$ value.
10. Take $h = 10^{-2}, 10^{-4}, \dots, 10^{-12}$ for the centered derivative of e^x at $x = 1$; observe the error *rise* again at very small h (round-off) and locate the best h .
11. Derive the centered-difference formula by subtracting the forward and backward Taylor expansions.
12. Estimate $f'(0.5)$ for $f(x) = \cos x e^{-x}$ with $h = 0.05$ (no exact derivative needed — discuss accuracy).

7 Experiment 5: Round-off Errors & Machine Epsilon

Objective

To determine the machine epsilon of the computer, to demonstrate round-off effects (loss of significance, non-associativity), and to observe the truncation/round-off trade-off that fixes an *optimal* step size.

7.1 Theory

Reals are stored in finite floating-point form $\pm(1.f) \times 2^e$. The **machine epsilon** is the gap between 1 and the next representable number,

$$\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16} \text{ (IEEE double),} \quad 2^{-23} \approx 1.19 \times 10^{-7} \text{ (single).}$$

Subtracting nearly-equal numbers causes *loss of significance*; round-off *grows* with the number of operations. The **total error** is truncation + round-off, and it is minimised at an optimal h (too large $\Rightarrow$ truncation dominates; too small $\Rightarrow$ round-off dominates).

Algorithm 5 Determine Machine Epsilon

Output: $\varepsilon_{\text{mach}}$

```

1:  $\varepsilon \leftarrow 1.0$ 
2: while  $(1.0 + \varepsilon/2.0) > 1.0$  do
3:    $\varepsilon \leftarrow \varepsilon/2.0$ 
4: end while
5: return  $\varepsilon$ 
```

$\triangleright = 2^{-52} \approx 2.22 \times 10^{-16}$ for double

7.2 Worked Example (full input and output)

Example 5.1 — machine epsilon and lost significance

Input. Run the algorithm above in double precision; also evaluate $1.0 + 10^{-20}$ and $(1 - \cos x)/x^2$ at $x = 10^{-4}$.

Output.

- $\varepsilon_{\text{mach}} = \boxed{2.22 \times 10^{-16}} = 2^{-52}$; the loop halves ε exactly 52 times.
- $1.0 + 10^{-20}$ stores as *exactly* 1.0 — the tiny addend falls off the end of the mantissa.
- $(1 - \cos x)/x^2$ direct = 0 (catastrophic cancellation, since $\cos 10^{-4}$ rounds to 1), whereas the true value is ≈ 0.5 . The stable form $\frac{2 \sin^2(x/2)}{x^2}$ gives 0.5 correctly.

Exercise 5 (Round-off & Machine Epsilon)

1. Compute $\varepsilon_{\text{mach}}$ for `float` (single) and `double` in your language; compare with 2^{-23} and 2^{-52} .
2. Show that $1.0 + \varepsilon_{\text{mach}} \neq 1.0$ but $1.0 + \varepsilon_{\text{mach}}/2 = 1.0$.
3. Evaluate $(1 - \cos x)/x^2$ at $x = 10^{-4}$ directly and via $2 \sin^2(x/2)/x^2$; explain the difference.
4. Sum $\sum_{k=1}^{10^6} 1/k$ in increasing *and* decreasing order of k ; compare the two totals.
5. Solve $x^2 - 1000x + 1 = 0$ by the naive quadratic formula and by the stable form $x_2 = c/(a x_1)$; compare the small root.
6. Find a, b, c for which $(a + b) + c \neq a + (b + c)$ in floating point.
7. For the centered derivative of e^x at $x = 1$, find the h that minimises the total error and compare with the rule $h_{\text{opt}} \sim \varepsilon_{\text{mach}}^{1/3}$.
8. Find the largest and smallest positive representable `double` (overflow/underflow limits).
9. Evaluate e^{-20} by its Maclaurin series (with alternating signs) and by $1/e^{20}$; explain the cancellation in the first.
10. How many significant digits are lost in $1.234567891 - 1.234567800$?
11. Demonstrate numerically that the round-off in a long dot product grows with the vector length N .
12. Tabulate (or plot) the total error of a finite-difference derivative versus $1/h$ and mark the optimal step.

Appendix: Quick Reference of the Foundations

Tool	Formula	Error	Used for
Euler's method	$v_{i+1} = v_i + \phi(t_i, v_i) h$	$O(h)$	solving an ODE model
Relative error	$\varepsilon_t = \left \frac{x_{\text{true}} - \hat{x}}{x_{\text{true}}} \right \times 100\%$	—	measuring accuracy
Stopping rule	stop when $\varepsilon_a < \varepsilon_s = 0.5 \times 10^{2-n}\%$	—	n significant figures
Taylor series	$f(x_{i+1}) = \sum_{k=0}^n \frac{f^{(k)}(x_i)}{k!} h^k + R_n$	$R_n = O(h^{n+1})$	approximating a function
Forward / backward diff.	$\frac{f_{i+1} - f_i}{h}, \frac{f_i - f_{i-1}}{h}$	$O(h)$	first derivative
Centered difference	$\frac{f_{i+1} - f_{i-1}}{2h}$	$O(h^2)$	first derivative
Machine epsilon	$\varepsilon_{\text{mach}} = 2^{-52}$ (double)	—	round-off limit

The big picture. A model becomes an equation; when it has no closed-form solution we approximate it numerically. Every numerical result then carries **truncation** error (from a finite process) and **round-off** error (from finite digits) — and the Taylor series is the common tool that quantifies the first and generates the formulae.